EnergyWatch AI

GESTION DES INCIDENTS

# Incident Runbook

Severity levels, general procedure and the four runbooks RB-001 to RB-004 applied by the monitoring agent.

| | |
|---|---|
| Projet | **EnergyWatch AI, plateforme multi-agents (LangGraph)** |
| Secteur | **Énergie** |
| Cadre | **Mini-projet S5 · Agent en production** |
| Version | **1.2.0** |
| Date | **30 octobre 2026** |
| Dépôt | **github.com/bouincha/Energy-Watch-AI** |

# Incident Runbook

The **monitoring agent** detects incidents automatically and applies the first steps of these runbooks. Incidents are visible in the dashboard (**Monitoring → Incidents**), at `GET /api/incidents`, and in the logs (`"logger": "energywatch.incidents"`).

The runbook IDs below match `app/core/incidents.py`.

## Severity levels

| Severity | Meaning | Target response |
|---|---|---|
| SEV-1 | Service down for all users | 15 min |
| SEV-2 | Major feature degraded (AI answers or data) | 1 h |
| SEV-3 | Partial or performance degradation, security noise | 1 business day |

## General procedure

1. **Acknowledge.** Open the incident in Monitoring and note its ID (`INC-xxxx`).
2. **Scope.** Click an affected correlation ID to see the agent waterfall of the failing run.
3. **Logs.** Run `docker logs energywatch | grep <correlation-id>`, or use Render → Logs and search for the ID.
4. **Mitigate** by following the matching runbook below.
5. **Resolve.** The incident closes automatically, or you can click **Resolve** and add a note, which is written to the audit trail.
6. **Post-mortem** for SEV-1 and SEV-2: copy `INCIDENT_REPORT_INC-0001.md` as a template.

---

## RB-001: LLM provider outage · SEV-2

**Trigger:** the LLM circuit breaker opens after `CIRCUIT_BREAKER_THRESHOLD` (3) consecutive failures.

**Symptoms:** answers carry the ⚠️ *Degraded mode* banner; the sidebar shows `LLM: … · open`; `/ready` returns `"status": "degraded"`.

| Step | Who | Action |
|---|---|---|
| 1. Detect | Monitoring agent (auto) | Receives the breaker OPEN event and opens the incident |
| 2. Mitigate | Agents (auto) | Switch to deterministic fallbacks. No LLM calls are made while the breaker is open. |
| 3. Communicate | System (auto) | Answers are flagged as degraded; the dashboard shows the open incident |
| 4. Diagnose | On-call | Check the provider status page and the API key/quota. Logs show `LLM call failed … error_type=…` |

| Step | Who | Action |
|---|---|---|
| 5. Recover | System (auto) | After `CIRCUIT_BREAKER_RESET_S` (30 s), the breaker goes HALF-OPEN and the next call probes the provider |
| 6. Resolve | System (auto) | The first successful call closes the breaker and the incident (MTTR recorded) |

**If the provider stays down:** switch provider with `LLM_PROVIDER=gemini` (or another) plus its key, and redeploy. As a last resort, use `LLM_PROVIDER=mock`.

## RB-002: Latency SLO breach · SEV-3

**Trigger:** a run exceeds `SLO_LATENCY_MS` (8,000 ms).

1. **Detect** (auto): the monitoring agent compares the run's duration with the SLO.
2. **Diagnose:** open the run's waterfall and find the slow agent. LLM agents are usually the cause.
3. **Mitigate:** check the provider latency, lower `LLM_TIMEOUT_S`, choose a faster model (`LLM_MODEL`), or switch provider.
4. **Resolve** (auto): the incident closes after 3 consecutive runs within the SLO.

## RB-003: Prompt-injection attack burst · SEV-3

**Trigger:** 3 or more blocked injection attempts within 10 minutes.

1. **Detect** (auto): the guardrails agent blocks each request and records a security event.
2. **Contain** (auto): blocked requests never reach the LLM or the tools (0 LLM calls on those runs).
3. **Investigate:** Monitoring → Run explorer → filter `blocked`. Look at the queries and the audit trail (`security.blocked`).
4. **Decide:** if the traffic is abusive, lower `RATE_LIMIT_PER_MINUTE` or block the client upstream.
5. **Resolve** (manual): click **Resolve** with a note describing what was reviewed.

## RB-004: Data tool failure · SEV-2

**Trigger:** an energy data tool raises an error during a run.

1. **Detect** (auto): the specialist catches the exception and reports it in its span.
2. **Mitigate** (auto): the run continues with the other agents, and the answer is flagged as partial or degraded.
3. **Diagnose:** look at the span error in the waterfall and the logs for that correlation ID. Check the data source.
4. **Resolve** (auto): closes on the next successful tool call.

---

## Game day: rehearsing the runbook

Use **Monitoring → Game day · chaos** (or `POST /api/chaos`) to inject failures safely:

| Toggle | Exercises |
|---|---|
| LLM provider outage | RB-001 (breaker, degraded mode, auto-recovery) |
| Data tool failure | RB-004 |
| Extra LLM latency | RB-002 |
| "Simulate injection burst" button | RB-003 |

The chaos endpoints require `X-API-Key` when `ADMIN_API_KEY` is set, and every change is audited.